

Problemi di ottimizzazione e ricerca

Sunday, April 26, 2026

5:33 PM

ESPLORAZIONE DELLO SPAZIO DELLE SOLUZIONI

- Approccio incrementale:
 - o Soluzione iniziale vuota
 - o Estensione della soluzione mediante applicazione di scelte
 - o Terminazione al raggiungimento di soluzione

Se l'algoritmo non conosce nulla del problema si dice non informato, se ha una conoscenza specifica si dice informato. Se l'algoritmo è in grado di esplorare l'intero spazio si dice completo.

Lo spazio delle soluzioni viene rappresentato come un albero di ricerca:

- Di altezza n , dove n è la dimensione del problema
- Di grado k , dove k è il numero massimo di scelte possibili
- La radice è la soluzione iniziale vuota
- I nodi intermedi sono le soluzioni parziali
- Le foglie sono le soluzioni.

CALCOLO COMBINATORI E SPAZIO DELLE SOLUZIONI

Il calcolo combinatorio rappresenta le regole di associazione che si basano su

- Gli elementi sono distinti
- Gli elementi sono ordinati
- Gli elementi sono ripetuti

Esso conta quanti sottoinsiemi di un insieme dato che godono di una certa proprietà, cioè determina il numero di nodi mediante i quali possono essere associati, secondo prefissate regole, gli elementi di uno stesso gruppo.

- Addizione: se un insieme di oggetti S è diviso in sottoinsiemi a 2 a 2 disgiunti tali che la loro unione sia S , il numero di oggetti in S può essere determinato sommando il numero di oggetti in ciascuno dei sottoinsiemi.
- Moltiplicazione: dati n insiemi S_i ciascuno di cardinalità $|S_i|$, il numero di n -uple ordinate è uguale alla produttoria delle cardinalità.
- Disposizioni semplici: una disposizione semplice di n oggetti distinti di classe k è un sottoinsieme ordinato composto da k oggetti degli n iniziali.

$D_{n,k} = \frac{n!}{(n-k)!}$; gli elementi sono tutti distinti e l'ordinamento conta

- Disposizione con ripetizioni: è un sottoinsieme ordinato composto da k degli n oggetti, ognuno dei quali può essere preso fino a k volte.

$D'_{n,k} = n^k$: l'ordinamento conta e ogni elemento può comparire nell'insieme per un massimo di k volte, in cui k può essere maggiore di n .

- Permutazioni semplici: è una disposizione semplice di n oggetti distinti di classe n . Si tratta di un sottoinsieme ordinato composto da n oggetti.

$P_n = D_{n,n} = n!$: l'ordinamento conta e ci sono esattamente n oggetti non ripetuti.

- Permutazioni con ripetizione: dato un multiinsieme di n oggetti di cui a uguali fra loro, b uguali fra loro ecc..., il numero di permutazioni distinte con oggetti ripetuti è:

$$P_n^{a,b,\dots} = \frac{n!}{a!b!\dots}$$

- Combinazioni semplici: una combinazione semplice di n oggetti distinti di classe k è un sottoinsieme non ordinato composto da k degli n oggetti. L'ordinamento non conta e non si possono avere ripetizioni. $C_{n,k} = \frac{D_{n,k}}{P_k} = \frac{n!}{k!(n-k)!}$
- Combinazioni con ripetizioni: è un sottoinsieme non ordinato composto da k degli n oggetti ognuno dei quali può essere preso fino a k volte. L'ordinamento non conta

nemmeno qua. $C'_{n,k} = \frac{(n+k-1)!}{k!(n-1)!}$

- Insieme delle parti: dato un insieme s di k elementi, il powerset e l'insieme dei sottoinsiemi di s (incluso s e l'insieme vuoto).
- Partizione: dato un insieme I di n elementi, una collezione s forma una partizione se e solo se:
 - o I blocchi di s non sono vuoti
 - o I blocchi sono a coppie disgiunte
 - o E la loro unione è I

PRINCIPIO DELLA MOLTIPLICAZIONE

```
def calcola_combinazioni_moltiplicazione(opzioni_per_categoria):
```

```
    """
```

```
    Calcola il numero totale di combinazioni basandosi  
    sul principio della moltiplicazione.
```

```
    """
```

```
    totale = 1
```

```
    for numero_scelte in opzioni_per_categoria:
```

```
        totale *= numero_scelte
```

```
    return totale
```

```
# Esempio basato sul menu: [2, 3, 2, 4]
```

```
menu = [2, 3, 2, 4]
```

```
risultato = calcola_combinazioni_moltiplicazione(menu)
```

```
print(f"Numero totale di pranzi possibili: {risultato}")
```

BACKTRACKING

Questa tecnica consiste nell'annullare l'ultima scelta effettuata al momento del ritorno da una funzione: così facendo la funzione sarà sicuramente non-tail recursive.

Il backtracking non deve necessariamente esplorare ogni ramo dell'albero di ricerca. Il pruning permette di tagliare i rami che sappiamo non porteranno a una soluzione valida, senza perdere la completezza della ricerca.

DISPOSIZIONI SEMPLICI

```
def disposizioni_semplici(pos, val, sol, mark, n, k, cnt):
```

```
    """
```

```
    pos: posizione corrente (da 0 a k-1)
```

```
    val: lista degli n elementi disponibili
```

```
    sol: lista per la soluzione parziale
```

```
    mark: lista booleana per marcare elementi già usati
```

```
    n: numero elementi in val
```

```
    k: classe della disposizione
```

```
    cnt: contatore soluzioni
```

```
    """
```

```
    if pos >= k:
```

```
        print(sol)
```

```
        return cnt + 1
```

```
    for i in range(n):
```

```
        if not mark[i]:
```

```
            mark[i] = True
```

```
            sol[pos] = val[i]
```

```
            cnt = disposizioni_semplici(pos + 1, val, sol, mark, n, k, cnt)
```

```
            mark[i] = False # Backtrack
```

```
    return cnt
```

In questo caso, prendi 2 elementi distinti da $\{1, 2, 3, 4\}$ e ne consideri l'ordine (quindi $(1, 2)$ è diverso da $(2, 1)$).

- **Le coppie possibili sono:**

- Partendo con 1: (1, 2), (1, 3), (1, 4)
- Partendo con 2: (2, 1), (2, 3), (2, 4)
- Partendo con 3: (3, 1), (3, 2), (3, 4)
- Partendo con 4: (4, 1), (4, 2), (4, 3)

Totale: $4 \times 3 = 12$ disposizioni.

DISPOSIZIONI CON RIPETIZIONI

```
def disposizioni_ripetute(pos, val, sol, n, k, cnt):  
    if pos >= k:  
        print(sol)  
        return cnt + 1  
  
    for i in range(n):  
        sol[pos] = val[i]  
        cnt = disposizioni_ripetute(pos + 1, val, sol, n, k, cnt)  
    return cnt
```

Esempio con $k=2$ (su insieme $\{1, 2, 3, 4\}$)

Mentre nelle disposizioni semplici le coppie come $(1, 1)$ o $(2, 2)$ erano proibite, qui sono permesse.

- **Le sequenze possibili sono:**

- Iniziando con 1: (1, 1), (1, 2), (1, 3), (1, 4)
- Iniziando con 2: (2, 1), (2, 2), (2, 3), (2, 4)
- Iniziando con 3: (3, 1), (3, 2), (3, 3), (3, 4)
- Iniziando con 4: (4, 1), (4, 2), (4, 3), (4, 4)

Totale: $4 \times 4 = 16$ (ovvero 4^2).

PERMUTAZIONI SEMPLICI

```
def permutazioni_semplici(pos, val, sol, mark, n, cnt):  
    if pos == n:  
        print(sol)  
        return cnt + 1  
  
    for i in range(n):  
        if not mark[i]:  
            mark[i] = True  
            sol[pos] = val[i]  
            cnt = permutazioni_semplici(pos + 1, val, sol, mark, n, cnt)  
            mark[i] = False # Backtrack  
    return cnt
```

Esempio: val = [1, 2, 3]

permutazioni_semplici(0, [1, 2, 3], [0]*3, [False]*3, 3, 0)

Esempio con $\{1, 2, 3\}$

Le permutazioni di $\{1, 2, 3\}$ sono:

$(1,2,3), (1,3,2), (2,1,3), (2,3,1), (3,1,2), (3,2,1)$.

Totale: $3! = 6$.

PERMUTAZIONI CON RIPETIZIONI

Si usano quando l'insieme contiene degli elementi duplicati.

```
def permutazioni_con_ripetizione(pos, val, sol, mark, n, cnt):
    if pos == n:
        print(sol)
        return cnt + 1

    # Per evitare duplicati, segniamo i valori già usati a questo livello
    usati_a_questo_livello = set()

    for i in range(n):
        if not mark[i] and val[i] not in usati_a_questo_livello:
            mark[i] = True
            usati_a_questo_livello.add(val[i])
            sol[pos] = val[i]
            cnt = permutazioni_con_ripetizione(pos + 1, val, sol, mark, n, cnt)
            mark[i] = False
    return cnt
```

Esempio con $\{1, 1, 2\}$

Se usassimo l'algoritmo delle permutazioni semplici, otterremmo:

$(1_a, 1_b, 2), (1_b, 1_a, 2), (1_a, 2, 1_b), \dots$ (risultati duplicati).

Con l'algoritmo sopra, le permutazioni uniche sono:

$(1, 1, 2), (1, 2, 1), (2, 1, 1)$.

Totale: $\frac{3!}{2!} = 3$.

COMBINAZIONI SEMPLICI

```
def combinazioni_semplici(start, val, sol, k, pos):
    if pos == k:
        print(sol)
        return 1

    count = 0
    # 'start' assicura che prendiamo solo elementi successivi,
    # evitando permutazioni duplicate (es. non rifacciamo {2,1} se abbiamo già {1,2})
    for i in range(start, len(val)):
        sol[pos] = val[i]
        count += combinazioni_semplici(i + 1, val, sol, k, pos + 1)
    return count
```

Esempio: `combinazioni_semplici(0, [1, 2, 3, 4], [0]*2, 2, 0)`

Esempio con $\{1, 2, 3, 4\}$ e $k=2$

Combinazioni Semplici (senza ripetizioni):

$\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}$.

(Nota: $\{2, 1\}$ non compare perché è uguale a $\{1, 2\}$).

Totale: 6.

COMBINAZIONI CON RIPETIZIONE

```
def combinazioni_con_ripetizione(start, val, sol, k, pos):
    if pos == k:
        print(sol)
        return 1
```

```

count = 0
# Qui passiamo 'i' invece di 'i + 1', permettendo il riuso dell'elemento
for i in range(start, len(val)):
    sol[pos] = val[i]
    count += combinazioni_con_ripetizione(i, val, sol, k, pos + 1)
return count

```

Combinazioni con Ripetizione: Include le precedenti più le coppie identiche: $\{1, 1\}$, $\{1, 2\}$, $\{1, 3\}$, $\{1, 4\}$, $\{2, 2\}$, $\{2, 3\}$, $\{2, 4\}$, $\{3, 3\}$, $\{3, 4\}$, $\{4, 4\}$. **Totale: 10.**

POWERSET

1) DIVIDI ET IMPERA

```

def powerset_divide_et_impera(s):
    if not s:
        return [[]]

    primo = s[0]
    resto = s[1:]

    # Sottoinsiemi senza il primo elemento
    senza_primo = powerset_divide_et_impera(resto)
    # Sottoinsiemi con il primo elemento
    con_primo = [[primo] + sotto for sotto in senza_primo]

    return senza_primo + con_primo

```

Esempio: powerset_divide_et_impera([1, 2, 3])

2) DISPOSIZIONI RIPETUTE

Ogni elemento viene trattato come un bit (0 o 1). Se il bit è 1, l'elemento è incluso, se 0 è escluso. Poiché ci sono 2 stati per ogni elemento, il powerset è equivalente a una disposizione con ripetizione di lunghezza n sull'alfabeto $\{0, 1\}$

```

def powerset_bitmask(val, pos, sol, n):
    if pos == n:
        # Crea il sottoinsieme basato sulla maschera sol
        sottoinsieme = [val[i] for i in range(n) if sol[i] == 1]
        print(sottoinsieme)
        return 1

    # Ramo 0: elemento escluso
    sol[pos] = 0
    count = powerset_bitmask(val, pos + 1, sol, n)
    # Ramo 1: elemento incluso
    sol[pos] = 1
    count += powerset_bitmask(val, pos + 1, sol, n)
    return count

```

3) COMBINAZIONI SEMPLICI

```

def combinazioni_semplici(start, val, sol, k, pos):
    if pos == k:
        print(sol)
        return 1
    count = 0

```

```

for i in range(start, len(val)):
    sol[pos] = val[i]
    count += combinazioni_semplici(i + 1, val, sol, k, pos + 1)
return count

```

```

def powerset_tramite_combinazioni(val):
    n = len(val)
    for k in range(n + 1):
        combinazioni_semplici(0, val, [0]*k, k, 0)

```

Per spiegare il flusso del **Powerset** di $\{1, 2, 3, 4\}$ utilizzando i tre approcci discussi, immagina di dover generare tutti i 16 sottoinsiemi ($2^4 = 16$).

Ecco come si comportano i tre algoritmi visivamente:

1. Divide et Impera (Approccio Ricorsivo)

Il flusso parte dal basso: divide l'insieme finché non arriva a quello vuoto, poi "risale" aggiungendo l'elemento che aveva isolato.

- **Logica:** $\text{Powerset}([1,2,3,4]) = \text{Powerset}([2,3,4]) + 1 + \text{Powerset}([2,3,4])$
- **Flusso:**
 - L'algoritmo crea un ramo per l'insieme vuoto [].
 - Risalendo, aggiunge 4: [], [4].
 - Poi aggiunge 3: [], [4], [3], [3, 4].
 - Continua così fino ad avere tutti i 16 sottoinsiemi.
- **Visivamente:** È un albero che si espande a ogni livello raddoppiando le soluzioni trovate al livello precedente.

2. Disposizioni Ripetute (Bitmask)

Qui il flusso è il più ordinato. Immagina di contare in binario da 0000 a 1111 (da 0 a 15).

- **Logica:** Ogni posizione $\text{sol}[i]$ rappresenta la presenza (1) o assenza (0) dell'elemento $\text{val}[i]$.
- **Flusso (i primi passi):**
 - 0000 -> []
 - 0001 -> [4]
 - 0010 -> [3]
 - 0011 -> [3, 4]
 - ...
 - 1111 -> [1, 2, 3, 4]
- **Visivamente:** È un albero binario completo di profondità 4. A ogni livello, il nodo si biforca: ramo sinistro (0) e ramo destro (1). È il metodo più "meccanico".

3. Combinazioni Semplici (Per dimensione)

Questo flusso non genera i sottoinsiemi in ordine casuale, ma li raggruppa per "quantità di elementi".

- **Logica:** Eseguiamo il ciclo k da 0 a 4:
 1. **k=0:** Scegli 0 elementi -> []
 2. **k=1:** Scegli 1 elemento -> [1], [2], [3], [4]
 3. **k=2:** Scegli 2 elementi -> [1,2], [1,3], [1,4], [2,3], [2,4], [3,4]
 4. **k=3:** Scegli 3 elementi -> [1,2,3], [1,2,4], [1,3,4], [2,3,4]
 5. **k=4:** Scegli 4 elementi -> [1,2,3,4]
- **Flusso:** L'algoritmo completa prima tutte le combinazioni di una certa lunghezza prima di passare a quella successiva.
- **Visivamente:** È un insieme di "livelli" separati. Immagina una piramide dove ogni piano contiene tutti i sottoinsiemi che hanno lo stesso numero di elementi.

Quale scegliere per l'insieme $\{1, 2, 3, 4\}$?

Metodo	Quando è migliore?
Divide et Impera	Se devi processare i dati in modo ricorsivo naturale (stile programmazione funzionale).

Bitmask Se devi enumerare **tutti** i sottoinsiemi velocemente: è il più rapido da scrivere e il più efficiente in memoria.

Combinazioni Se hai un vincolo di **dimensione** (es. "voglio solo i sottoinsiemi di 2 elementi"), non ha senso generare tutto il powerset.

PARTIZIONE

1) DISPOSIZIONI RIPETUTE

```
def partizioni(indice, n, blocchi_correnti):  
    # indice: elemento che stiamo inserendo (1..n)  
    # blocchi_correnti: lista di liste  
    if indice > n:  
        print(blocchi_correnti)  
        return
```

```
# 1. Prova a inserire in un blocco esistente  
for i in range(len(blocchi_correnti)):  
    blocchi_correnti[i].append(indice)  
    partizioni(indice + 1, n, blocchi_correnti)  
    blocchi_correnti[i].pop() # Backtrack
```

```
# 2. Prova a creare un nuovo blocco  
blocchi_correnti.append([indice])  
partizioni(indice + 1, n, blocchi_correnti)  
blocchi_correnti.pop() # Backtrack
```

L'esempio proposto nel materiale didattico mostra come le partizioni vengano raggruppate per numero di blocchi k :

- **$k=1$** : $\{1, 2, 3, 4\}$ (un unico grande blocco).
- **$k=2$** : Ci sono 7 possibili partizioni, ad esempio:
 - $\{1, 2, 3\}, \{4\}$
 - $\{1, 2\}, \{3, 4\}$
 - $\{1\}, \{2, 3, 4\}$ (e altre)
- **$k=3$ e $k=4$** : Si procede suddividendo ulteriormente gli elementi.

2) ALGORITMO DI ER

L'idea fondamentale è: dato un elemento n , esso può essere inserito in uno dei blocchi già creati oppure può dare inizio a un **nuovo blocco**.

```
def genera_partizioni(elemento, n, blocchi):  
    # 'elemento': numero che stiamo attualmente posizionando (1..n)  
    # 'blocchi': lista di liste, dove ogni lista è un blocco  
  
    if elemento > n:  
        # Abbiamo posizionato tutti gli elementi, stampiamo la partizione  
        print(blocchi)  
        return
```

```
# 1. Prova a inserire l'elemento in ogni blocco già esistente  
for i in range(len(blocchi)):  
    blocchi[i].append(elemento)  
    genera_partizioni(elemento + 1, n, blocchi)  
    blocchi[i].pop() # Backtracking
```

```
# 2. Prova a creare un nuovo blocco con questo elemento  
blocchi.append([elemento])  
genera_partizioni(elemento + 1, n, blocchi)  
blocchi.pop() # Backtracking
```

```
# Esempio per n=3: genera_partizioni(2, 3, [[1]])
```